

INF 213 - Roteiro da Aula Prática 1

Arquivos fonte e diagramas utilizados nesta aula:

<https://drive.google.com/file/d/1127LdexMcuTr1H0Qc5zJ8WanNwrmlYaa/view?usp=sharing>

Como nesta prática há algumas perguntas, faça uma cópia deste documento e acrescente as respostas nela.

Etapa 1

Considere o programa “complexidade1.cpp”. Ele possui 5 funções, sendo que cada uma delas realiza uma quantidade diferente de somas.

Estude brevemente o código, compile-o e, a seguir, faça as atividades abaixo (o programa deve ser executado usando a sintaxe “./a.out N”, onde N é o “tamanho da entrada”):

a) Meça o tempo de execução para diferentes tamanhos de entrada: 1, 2, 3, 4, 10, 11, 12, 13, 14, 50, 100, 500, 1000 (apenas entenda e observe os tempos -- não é preciso anotá-los aqui). Não meça (agora) para valores maiores que 1000.

b) Meça os tempos para $n=1000$ e adicione-os à tabela abaixo. A seguir, tente ADIVINHAR o tempo para $n=2000$ (adivinhe antes de testar!!!) com base nos resultados para $n=1000$. Finalmente, acrescente na última coluna o tempo (medido pelo programa para $n = 2000$). (não se preocupe -- você não perderá pontos se errar agora....)

N	1000 (medida)	2000 (adivinhada)	2000 (medida)
Funcao0	0.0177030	0,070812	0,0176190
Funcao que executa n somas	0.0028730	0,011492	0,0052910
Funcao que executa n^2 somas	2.2387060	8,954824	9,4038480
Funcao que executa $2n^2$ somas	2.2487790	9,95116	9,3609090
Funcao que executa $4n^2 + n$ somas	2.0551820	8,220728	8,1875940

c) O que podemos concluir sobre o tempo nas diferentes funções para valores pequenos de n ? (1, 2, 3, 10, 11, ...)

É possível concluir que quando os valores são pequenos, a variação do tempo é mínima. As variações consideráveis são vistas a partir do $n= 100$.

d) O que podemos concluir sobre as diferenças de tempo para valores maiores de N? (na verdade esse experimento simples não é suficiente para garantir que essa conclusão é correta -- mas ela é! Com o tempo veremos que isso é algo que realmente ocorre na prática)

Pode-se concluir que a diferença de tempo para valores maiores de n aumentam mais rápido do que para valores menores

e) Na função 1, por que a soma do ct é mais “importante” (crítica para analisar o algoritmo) do que o “i++”?

É mais importante pois em geral, a operação mais interna do loop é considerada mais importante, mas por se tratar de operações com inteiros, a soma do ct tem a mesma complexidade que o i++.

Etapa 2

Considere o programa “complexidade2.cpp”. Ele possui 4 funções, sendo que cada uma delas realiza uma quantidade diferente de operações.

a) Qual a operação básica em cada um delas?

funcao1	ct += i ;
funcao2	ct += i + j;
funcao3	ct += i + j;
funcao4	ct += i ;

b) Quantas vezes a operação básica é realizada em cada um ? (no caso da funcao4, pode ser uma resposta aproximada)

funcao1	N vezes
funcao2	N ² vezes
funcao3	$\frac{N(N+1)}{2}$ vezes
funcao4	N vezes

c) Meça o tempo de execução de cada função para os diferentes valores de N abaixo (coloque-os na tabela):

N	10	20	50	100	200
funcao1	0,0000560	0,0000420	0,0003590	0,0008800	0,0011560
funcao2	0,0005660	0,0007740	0,0063100	0,0260550	0,0971060
funcao3	0,0019880	0,0055120	0,0999290	0,7784270	5,9586490
funcao4	0,0086150	0,0178590	0,0481260	0,1150770	0,2708100

d) O que podemos concluir sobre os tempos ?

É possível concluir que nas funções 1,2 e 4 o tempo aumenta proporcionalmente com o aumento do N, porém, na função 3 o tempo não cresce linearmente, e sofre uma oscilação, devido à sua complexidade ser $\frac{N(N+1)}{2}$ vezes.

Etapa 3

Compile o programa anterior utilizando a flag “-O3” do g++ (g++ -O3 complexidade2.cpp). Meça novamente os tempos para N = 200.

Essa flag ativa o nível máximo de otimização do compilador. Ela normalmente não reduz a complexidade dos algoritmos, mas acelera de forma considerável o tempo de processamento ao realizar diversos tipos de otimizações (nesta etapa não há nada a ser entregue/respondido)

Etapa 4

Considere o programa complexidade3.cpp . Veja o código-fonte e entenda o que ele faz.

A seguir, compile o programa e teste-o com o arquivo de teste entrada_5.txt (./a.out < entrada_5.txt > saida.txt). Para ver a saída, basta digitar “cat saida.txt” (no Linux)

Concentre-se apenas na função “encontraPosicoes” (nesta prática NÃO altere nada em outras partes do programa -- especialmente a parte que executa sua função 1 milhão de vezes).

Código original

Qual a operação básica da função encontraPosicoes?

numeros[j]==i;

Quantas vezes essa operação é executada para uma entrada de tamanho N?

N^2 vezes.

Teste o desempenho do programa considerando os arquivos de teste disponibilizados (os números representam o tamanho da entrada)

Primeira melhoria

Note que, quando encontramos a posição de um determinado número “i”, não precisamos continuar procurando por esse número no array (podemos parar e passar para o próximo número). Modifique seu programa utilizando essa estratégia para melhorar sua performance.

Considerando a nova versão do programa:

Quantas vezes a operação básica é executada para uma entrada de tamanho N?

N vezes

Teste o desempenho do programa considerando os arquivos de teste disponibilizados (os números representam o tamanho da entrada) Como esse tempo se compara com o obtido na versão original? **Em todos os arquivos de entrada, o tempo de execução caiu pela metade.**

Segunda melhoria

Dada uma entrada pequena (por exemplo, os números 2,0,1,3,4,5) encontre a saída (para o problema tratado neste exercício) **utilizando uma folha de papel**. Pense em uma forma mais eficiente de resolver o problema e modifique “encontraPosicoes” para funcionar utilizando essa nova estratégia. Sua nova função deverá ficar MUITO mais eficiente (para qualquer tamanho de entrada, principalmente as maiores).

Considerando a nova versão do programa:

Quantas vezes a operação básica é executada para uma entrada de tamanho N?

Teste o desempenho do programa considerando os arquivos de teste disponibilizados (os números representam o tamanho da entrada), principalmente os maiores. Como esse tempo se compara com o obtido na versão original?

Submissao da aula pratica:

A solucao deve ser submetida até o deadline informado no sistema submittity (submittity.dpi.ufv.br).

Observe que o submittity corrige seu trabalho instantaneamente! Confira com atenção sua nota até o deadline!! (ela não será alterada depois!)

Deverão ser enviados (não envie mais arquivos .cpp):

- 1) Um PDF deste documento (contendo suas respostas para as perguntas), com nome roteiro.pdf
- 2) A versão final do programa complexidade3.cpp (ou seja, a versão com a segunda melhoria).

Os nomes dos arquivos devem ser EXATAMENTE esses. Se o nome estiver errado sua nota ficará 0. (isso poderá ser visto poucos segundos após a entrega e corrigido até o deadline)

Cada envio sobrescrever o anterior. Ou seja, após terminar a última etapa é preciso enviar os arquivos de todas entradas!